Higher-Order Matchers

All the matchers seen so far are primitives. Now, we're going to look at higher-order matchers—that is, matchers that you can pass other matchers into. With this technique, you can build up composed matchers that specify exactly the behavior you want.

Collections and Strings

One of the primary tasks of programming, in any language, is dealing with collections, and Ruby is no exception. RSpec ships with six different matchers for dealing with data structures:

- include requires certain items to be present (in any order).
- start_with and end_with require items to be at the beginning or end.
- all checks a common property across all items.
- match checks a data structure against a pattern.
- contain_exactly requires certain items, *and no others*, to be present (in any order).

All of these matchers also work with strings, with a few minor differences (after all, strings are just collections of characters!). In the following sections, we'll review the matchers in detail.

include

The include matcher is one of the most flexible, useful matchers RSpec provides. It's also a key defense against brittleness. By using include rather than a stricter matcher like eq or match, you can specify just the elements you care about. The collection can contain unrelated items, and your tests will still pass.

At its simplest, the include matcher works on any object with an include? method. Strings and arrays both support this method:

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb

```
expect('a string').to include('str')
expect([1, 2, 3]).to include(3)
```

For hashes, you can check for the presence of a specific *key* or *key-value pair* (passed as a hash):

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb

```
hash = { name: 'Harry Potter', age: 17, house: 'Gryffindor' }
expect(hash).to include(:name)
expect(hash).to include(age: 17)
```